

Governança em IA: Um Plano de Ação para Manutenção Preditiva via Prognóstico de Falhas em Máquinas via Séries Temporais Multi-Sensor

André Luiz Moreira Dutra

1. Introdução

O objetivo deste trabalho é definir um plano de ação que aborde o problema de detecção de falhas em equipamentos utilizando dados multi-sensor e modelos preditivos. O foco do trabalho será expor potenciais tomadas de decisão no âmbito de um cientista de dados em todas as etapas do processo, desde o diagnóstico do problema à implantação final, em um contexto de equipe.

O trabalho está dividido em seis partes: a Introdução, onde o problema e o objetivo do trabalho são apresentados; a Metodologia, onde a metodologia de criação do plano de ação é descrita; o Plano de Ação, que apresenta de maneira generalizada as ações e decisões a serem tomadas em conjunto com a equipe para a abordagem do problema; a Discussão, onde partes do plano de ação mais relevantes para o cientista de dados são executadas (se possível) ou são apresentadas com um roteiro mais detalhado voltado à abordagem do problema proposto; e a Conclusão, que apresenta as considerações principais sobre a solução proposta no trabalho, finalizando com a Bibliografia das citações.

2. Metodologia

A metodologia de tomada de decisão utilizada será o Domino Data Science Life Cycle[2], um guia de ações que cobre todo o processo de produção de produtos envolvendo ciência de dados, desde sua concepção ao deploy do produto de software ao usuário e seu devido monitoramento. A metodologia foi escolhida devido à sua versatilidade, completude e adaptação a metodologias ágeis atuais de desenvolvimento, embora outras abordagens como CRISP-DM[3] e TDSP[5] também apresentem resultados promissores na indústria.

A metodologia Domino divide o processo de produção em seis etapas: Ideação, Aquisição e Exploração dos dados, Pesquisa e Desenvolvimento, Validação, Entrega e Monitoramento. Cada etapa será explicada e detalhada no plano de ação a seguir. A

ideia desta organização não é observar o processo de maneira sequencial, como um modelo em cascata, e sim organizar o processo em ordem de precedência e prioridade. Porém iterações e retornos em etapas são esperados e incentivados, desde que contribuam para o andamento do processo.

3. Plano de Ação

A tabela seguinte apresenta o plano de ação para abordagem do problema. Na coluna à esquerda, temos as etapas da metodologia proposta, e à direita temos as ações a serem tomadas em cada etapa. Ações grifadas na coluna à direita serão tratadas com mais detalhe em relação ao problema específico na próxima sessão. Por se tratar de um plano em equipe, os agentes envolvidos em cada ação estão descritos e codificados por cores para organização, com a legenda na coluna esquerda.

1. IDEIAÇÃO Objetivo: Realizar o entendimento do problema pela perspectiva do cliente, definir requisitos e viabilidade do projeto e propor uma solução inicial. ● Cientista de dados ● Engenheiros de software e dados e desenvolvedores ● Técnicos em Manutenção Preditiva ● Representante do Produto (PO) e Business	● <u>Identificar o problema;</u> ● ● ● ● ● Definir o escopo do problema: ○ <u>Revisão da Literatura;</u> ● ○ Calcular ROI e valor do projeto em conjunto com o Business. ● ● ● Analisar viabilidade do projeto: ○ Definir viabilidade de implementação das soluções; ● ● ○ Definir viabilidade e método de obtenção dos dados necessários para o treino e execução dos modelos; ● ● ● ○ Manter uma documentação dos requisitos de negócio e métricas de sucesso definidas; ● ● Caso o processo seja inviável, abandonar projeto; ● Caso não seja prioritário, administrar backlogs para quando o projeto seja viável futuramente; ● ● ● ● Em caso de viabilidade, selecionar a representação inicial da solução a ser seguida e iniciar. ● ● ● ●
2. AQUISIÇÃO E EXPLORAÇÃO DE DADOS Objetivo: Adquirir dados, avaliar qualidade e conformidade com as soluções propostas. Identificar padrões iniciais e preparar dados para experimentação. ● Cientista de dados	● Caso os dados existam: ○ Identificar fontes e existência de pipelines ativas de coleta; ● ● ○ Sinalizar a equipe de engenharia de dados em caso de dificuldades de conexão. ● ● ● Caso os dados não existam: ○ Comprar ou gerar dados via experimentos em máquinas reais (conforme decidido na etapa anterior). ● ● ● ● ● Coletar os dados das fontes definidas; ● ● ● <u>Análise exploratória dos dados;</u> ● ● ● Pré-processar dados ●

- Engenheiros e devs - Técnicos em MP - PO e Business	Gerar datasets padronizados
3. PESQUISA & DESENVOLVIMENTO Objetivo: Selecionar, testar e validar soluções utilizando os dados da etapa anterior e aplicando o método científico. - Cientista de dados - Engenheiros e devs - Técnicos em MP - PO e Business	- Organização das hipóteses levantadas: - Descartar ou adaptar hipóteses incompatíveis com etapas anteriores; - Nova revisão da literatura em caso de identificação de novos problemas; - Ordenação de hipóteses por simplicidade (Navalha de Ockham) em relação aos requisitos. - Selecionar uma hipótese; - Definir o ambiente de experimentação, máquinas (físicas e virtuais) e demais ferramentas de experimentação com a equipe de engenharia de dados; - Realizar experimentos: - Divisão do dataset em treino, validação e teste; - Realizar validação cruzada se for viável; - Treino, validação e teste dos modelos. - Gerar dashboards de insight sobre os resultados: - Definir visualizações interativas e de fácil compreensão por outros membros da equipe; - Compartilhar insights ciclicamente com a equipe; - Levar em consideração as métricas de sucesso (KPIs) do projeto nos insights; - Gerar insights de validação em caso de sucesso do experimento; - Em caso de falha, coletar mais dados (se necessário e possível) e reiniciar o processo de pesquisa com uma nova hipótese.
4. VALIDAÇÃO Objetivo: Validar resultados entre todos os stakeholders. - Cientista de dados - Engenheiros de TI - Técnicos em MP - PO e Business	- Validar conformidade dos resultados com os requisitos dos stakeholders: - Atende aos requisitos de negócio? - É compatível com os casos de uso? - É compatível com a solução de software planejada? - Está pronto para deploy? - Retornar à pesquisa e desenvolvimento em caso de não-validação. - Publicar modelo.
5. ENTREGA Objetivo: Realizar a implementação, testagem e entrega do produto de	- Planejar a entrega com a equipe de desenvolvimento conforme metodologia ágil utilizada; - Prestar apoio à equipe de desenvolvimento de software no deploy do produto: - Documentar o modelo em relação à implementação e casos de uso; - Identificar todas as dependências de linguagem, bibliotecas,

software que configura a solução final. ● Cientista de dados ● Engenheiros de TI ● Técnicos em MP ● PO e Business	pacotes, ferramentas e dados utilizados; ○ Gerenciar versionamento dos dados e parâmetros do modelo; ○ Promover o isolamento do ambiente de execução do modelo para segurança e redução de riscos; ○ Preservar ferramentas, dashboards e relatórios úteis para o monitoramento. ● Gerenciar coletas de dados de tempo-real do produto e dos usuários: ●● ○ Garantir segurança e conformidade da coleta com a LGPD; ○ Definir pipelines de coleta de métricas de uso e de desempenho do modelo em tempo real; ○ Auxiliar na modelagem de testes A/B de usuário, caso haja. ● Validação com usuários e entrega do produto. ●
6. MONITORAMENTO Objetivo: Monitorar o desempenho da solução no contexto real de uso. ● Cientista de dados ● Engenheiros de TI ● Técnicos em MP ● PO e Business	● Monitorar ciclicamente as métricas de tempo real: ○ Métricas de usabilidade; ●● ○ Métricas de performance e desempenho; ●● ○ Métricas de segurança; ●●● ○ KPIs e métricas de valor. ●● ● Comparar com grupo controle caso houver, neste caso com a manutenção preditiva feita sem o auxílio dos modelos; ● ● Avaliar valor da solução para a empresa; ● ● Identificar pontos de melhoria e reiniciar o ciclo. ●●

4. Discussão

O objetivo desta seção é apresentar de maneira mais detalhada as principais etapas do processo relacionadas ao cientista de dados, especialmente na busca da solução do problema proposto. Se possível (como no caso da revisão da literatura) a ação será executada. Se não (como no caso da análise de dados) será listado um roteiro de perguntas que se buscaria responder nesta etapa, e o que fazer, quem consultar e que ações tomar para respondê-las.

4.1. Identificação do problema

A identificação do problema é a etapa mais importante de todo o ciclo de vida de ciência de dados, pois é o ponto que definirá o rumo, objetivo e os impactos esperados de todas as outras etapas do processo. O ponto mais importante desta etapa é que ela seja completamente voltada à definição do problema segundo as necessidades do cliente e dos usuários finais do sistema. Esta vertente é também a mais defendida em processos de engenharia de software e experiência do usuário [6]. Principalmente considerando o atual hype sobre modelos IA e principalmente voltados a deep-learning, é importante tomar uma postura que repreenda ativamente o design voltado aos dados e modelos, que não necessariamente codifica as necessidades reais.

Por esse motivo, esta etapa deve envolver todos os stakeholders do processo. A ideia é que o cliente, os desenvolvedores e o cientista de dados proponham requisitos para a solução de trás para frente, partindo do ponto final, o usuário, passando pelo ponto de contato direto com o usuário, a solução de software, para então atingir o ponto mais inicial, que é a definição do modelo de dados.

Com relação ao problema dado de previsão de falhas no equipamento utilizando dados multisensor, a definição dada é extremamente ampla e não reflete detalhes dos casos de uso da solução final, por isso vários pontos ainda devem ser esclarecidos.

Primeiramente, precisamos esclarecer com o representante do cliente os casos de uso da solução:

- Qual é o perfil do usuário final? Técnicos de MP? Operadores com menor especialização?
- Em que contexto a solução será usada? Uma ferramenta de monitoramento? Como parte de outras modelagens logísticas?
- Quais são as principais necessidades do usuário a serem atendidas? O resultado buscado é bem definido ou requer pesquisa também?
- Alguma solução já é empregada atualmente para o problema? Qual? Como ela é implementada?

Em seguida, devemos discutir sobre a modelagem da equipe de software para o problema:

- Já existem soluções em software para o problema na empresa ou seria necessária uma nova definição?
- As soluções existentes utilizam modelos de data science? Posso ter acesso à documentação?
- Caso não existam soluções, a modelagem é trivial ou amplamente conhecida na indústria? Qual seria?
- Caso não seja trivial, é possível ao menos definir o padrão de resposta esperado pelo modelo de ciência de dados?
- Existem outras soluções da empresa que poderiam reaproveitar este modelo de dados? Devemos realizar o desenvolvimento com isso em mente? Podemos assimilar novos dados?

O objetivo é tentar realizar a conexão entre a implementação do modelo e a solução de software empregada, principalmente se ela já existir. Com isso, podemos realizar a implementação da solução sem refazer software que já atendia ao cliente antes. Podemos partir agora para perguntas mais específicas sobre o modelo, que podem ser respondidas também com o auxílio de técnicos de manutenção preditiva. Primeiramente com relação à saída (o target), que é justamente o resultado do modelo que será usado pelas demais partes:

- O que queremos dizer com “prever falhas no equipamento”? Detecção de falhas presentes? Previsão de falhas futuras?
- Os tipos de falhas precisam ser detectados/previstos também?
- Com que frequência a saída precisa ser inferida?
- A informação que desejamos prever pode ser inferida dos relatórios de maneira trivial? É parte da pesquisa a inferência desta informação também?

Com estas informações podemos ter uma noção um pouco melhor de que tipo de modelo usar. Por exemplo, se a previsão precisar ser feita na faixa de no mínimo dias, por exemplo, um modelo LSTM aplicado diretamente aos dados não funcionaria, já que ele só tem capacidade de previsão na casa de 1000 valores futuros, que correspondem a menos de um dia em segundos. Também podemos definir a viabilidade de modelos dados os dados presentes. Vamos estender as perguntas ao formato de entrada e dados de treino do modelo, dessa vez direcionadas principalmente à equipe de engenharia de dados e os técnicos em MP:

- Como é coletada a entrada dos sensores? Já existem pipelines de dados para a coleta?
- Com que frequência a entrada é coletada? A frequência é compatível com a frequência de saída esperada? O overhead de comunicação entre a coleta e o modelo pode ser definido? E entre o modelo e o usuário final?
- Já existem dados de treino? Os dados refletem os casos de uso do cliente? É trivial modificá-los ou filtrá-los para que reflitam ou isso deve ser pesquisado?
- Caso os dados pertençam aos clientes, eles estão em conformidade com a LGPD? A implementação dos pipelines de dados é segura? Qual é o impacto do vazamento destes dados para o cliente e para a empresa?
- Caso os dados não existam, como serão obtidos? Via experimentos controlados em máquinas (run to failure)? Compra de dados? Um plano de ação pode ser definido para estes casos?

A existência e o custo de obtenção de dados são muitas vezes o principal gargalo de viabilidade para o desenvolvimento de soluções em software. Além disso, requisitos de segurança têm se tornado cada vez mais relevantes após a instituição da LGPD. Isso é especialmente importante neste caso porque um dos principais pontos de vulnerabilidade de sistemas atualmente está em sensores IoT instalados sem segurança. Por fim, tentamos definir inicialmente requisitos de desempenho para o modelo, caso hajam:

- Qual é o contexto de execução do modelo? Há recursos computacionais limitados? Tempo de resposta precisa ser curto?
- Métricas de desempenho mínimas para o modelo podem ser definidas?
 - É um modelo de alto risco? Qual é a gravidade de decisões erradas do modelo?

- Qual a margem de erro máxima aceita para a previsão?
- Qual o tempo mínimo no futuro a previsão deve ser feita para que seja viável? Por exemplo, na faixa de segundos, minutos ou dias?
- Há outras métricas de desempenho mais alinhadas com o caso de uso?
- É possível estimar as economias ou custos da empresa por caso preditivo? Quanto é economizado por uma previsão correta de falha? E por uma previsão correta de máquina saudável? Quanto é o gasto de previsões incorretas?

Com estas informações, podemos ter um conhecimento maior do problema e com isso pensar em soluções voltadas ao cliente final.

4.2. Revisão da Literatura

Dada a definição do problema, o próximo passo da ideação é realizar uma revisão da literatura. O objetivo desta revisão inicial não é definir o modelo final que será usado, e sim definir:

1. Se o problema já foi definido e estudado antes;
2. Por quais pontos de vista ele é usualmente abordado;
3. Quais as principais soluções propostas na literatura para o problema.

No contexto de ciência de dados, estes três pontos se traduzem para:

1. A definição formal do problema;
2. As tarefas de aprendizado de máquina/modelagem estatística utilizadas para modelar o problema;
3. As opções de modelos de machine learning/estatísticos mais promissores e eficientes para solucionar as tarefas.

Esta etapa é essencial para que o cientista de dados compreenda a complexidade do problema e das soluções necessárias para abordá-lo, a quantidade, tipo e qualidade dos dados necessários para o processo de ciência de dados e, com isso, poder definir a viabilidade do processo e ter uma noção mínima de requisitos necessários. Além disso, esta etapa visa criar um pool inicial de hipóteses de modelagem a serem testadas na terceira etapa, de pesquisa e desenvolvimento.

Revisão da Literatura (Execução)

Início a pesquisa buscando por artigos de revisão da literatura em machine learning para manutenção preditiva, já que o problema proposto é comum o suficiente para que sua definição formal e principais tarefas apareçam em artigos de revisão. Dentre os resultados, o artigo [4], que revisa modelos de deep learning aplicados a manutenção preditiva, traz as seguintes definições:

1. Fault Prognostics (Prognóstico de Falhas): Predição de falhas futuras em um sistema ou equipamento;
2. Fault Diagnostics (Diagnóstico de Falhas): Detecção e definição de falhas presentes em um sistema ou equipamento;
3. Sensor Forecasting (Previsão de Sensores): Predição de valores futuros de um sensor.

A definição 1 se traduz exatamente ao problema proposto, enquanto o artigo propõe que 2 e 3 podem ser combinadas para gerar um prognóstico.

Busco em seguida por artigos de revisão utilizando machine learning clássico para o problema de prognóstico de falhas. Segundo [8], uma revisão de técnicas de prognóstico voltadas a dados, a maior parte das pesquisas voltadas a FP modelam o problema através da tarefa de RUL Prediction, que é a predição da duração após a qual o sistema se modifica a um estado indesejado (Reamaining Useful Life - Vida Útil Remanescente). A ideia é estimar, dado o estado atual do equipamento, quanto tempo de vida ele tem até uma falha. Ambos os artigos classificam todos os modelos que abordam estes três problemas de maneira supervisionada nas seguintes tarefas:

1. Regressão sequencial (para RUL Prediction/Fault Prognosis);
2. Forecasting de séries temporais (para Sensor Forecasting);
3. Classificação sequencial (para Fault Diagnostics);

Os três são problemas semelhantes em ciência de dados: a ideia é, dada uma sequência de valores, prever algum valor correspondente à posição seguinte da sequência (Seja o RUL, os valores dos próximos sensores ou o tipo de falha). O protocolo usual nestes casos é ou aplicar janelas deslizantes na sequência de entrada, extrair features da janela e utilizar modelos supervisionados convencionais para prever os valores, ou aplicar modelos de deep learning sequenciais diretamente (estas duas opções também abrangem todas as soluções propostas em [4] e [8]).

Quanto aos modelos a serem utilizados, o artigo [10], que apresenta uma revisão atualizada (2 de abril de 2025) de modelos de deep learning aplicados a manutenção preditiva, aponta um tradeoff entre as principais estratégias: estratégias baseadas em engenharia de features funcionam com menos dados e recursos, porém requerem alto conhecimento das variáveis, vários testes de hipótese para modelagem das features e geram modelos extremamente específicos e sensíveis a ruído nos dados. Modelos baseados em deep learning, por sua vez, são mais generalistas, podem ser reutilizados em outras aplicações, são menos sensíveis a ruído e exigem menos engenharia de features, porém requerem muitos dados e recursos computacionais para atingir desempenho aceitável.

Tendo isto em vista, o artigo lista o Chronos [7] como o atual modelo estado-da-arte para forecasting de séries temporais via modelos pré-treinados

(foundation). O modelo é baseado em LLMs encoder-decoder e pré-treinado para realizar forecasting generalizado de séries temporais, apresentando ótimos resultados via fine-tuning e resultados consideráveis mesmo zero-shot (sem necessidade de treino, apenas da sequência anterior). Ainda esta semana (20 de outubro de 2025) foi lançado o Chronos 2 [1], uma atualização open source do Chronos que apresenta suporte para dados multivariados, completamente documentada e de fácil uso. O framework é open-source e permite também acesso aos embeddings do encoder e do decoder do modelo, que permitem a aplicação do modelo não só para forecasting, como também para classificação e regressão. Por ser um modelo pré-treinado, é um modelo que apresenta alto desempenho sem a alta necessidade de dados e recursos habitual de modelos de deep-learning, representando a opção mais viável e promissora de abordagem. Caso esta linha de desenvolvimento não seja promissora, deve-se avaliar entre os dois outros paradigmas de solução citados qual mais se alinha aos requisitos de negócio e recursos disponíveis, e com isso explorar soluções específicas.

Por fim, com relação ao formato sequencial esparsa das amostras, não foi encontrado nenhum artigo que utilizasse esta modelagem teórica ou prática através das buscas. Foram usadas keywords como "sequential multi-sensor data" "sequential sparse time-series data" "multivariate sparse time-series data" e nenhum dos artigos correspondia ao caso, tanto em ciência de dados quanto em outras áreas. O artigo mais próximo analisava séries temporais univariadas esparsas e realizavam a reconstrução via ou interpolações matemáticas ou modelos de machine learning [9], porém os modelos requerem variância na frequência de amostragem para a reconstrução, o que não é o caso dos dados presentes. Algumas opções são ou utilizar os dados de todos os sensores sequencialmente, como um sensor único, ou fundir os dados em blocos ou janelas contendo uma ou mais amostragens de cada sensor ao longo de cada sequência seguida de segundos, ambas compatíveis com todos os modelos apresentados.

Observações:

- O termo "Fault Prediction" (Predição de Falhas) parece ser usado de maneira ambígua na literatura para prognóstico ou diagnóstico de falhas. Neste trabalho foi considerada a interpretação como "Prognóstico de Falhas", porém esta questão teria sido desambiguada na etapa anterior em um contexto real.
- A falha na busca por artigos em dados sequenciais de sensores pode ter ocorrido por um desconhecimento de nomenclatura específica. Este é outro ponto de esclarecimento com stakeholders, principalmente técnicos em manutenção preditiva.

4.3. Análise Exploratória e Modelagem dos Dados

Como optamos inicialmente pelo uso do Chronos 2, que não requer muito pré-processamento da entrada, o foco da análise será em avaliar a qualidade e

quantidade dos dados e em preparar as features. Após a coleta dos dados, avaliamos inicialmente:

- **Consistência:** Qual é a taxa de registros repetidos? Remoção de duplicatas é trivial?
- **Integridade:** Qual é a taxa de registros faltantes? Como tratar sequências incompletas de sensores? Mesmo em integridade perfeita, como tratar a esparsidade dos dados?
- **Robustez:** Há muito ruído nos dados? Como o ruído nestes dados é tratado geralmente?
- **Tipo:** Todos os dados estão em tipagens compatíveis com os modelos hipotetizados? Como realizar a conversão?
- **Quantidade:** Quantas amostras restaram após o tratamento dos dados? É plausível considerar outras formas de tratamento que excluam menos dados?

A consistência dos dados pode ser tratada trivialmente, com uma simples remoção de duplicatas. Já a integridade é mais complexa, porque completar os dados faltantes envolve interpolar sequências de registros que já são esparsos. Por outro lado, apenas remover o registro do sensor faltante só seria plausível em modelagens usando janelas deslizantes, onde a ausência de algumas features seria pouco prejudicial em um contexto onde médias (ou outras métricas) de features em múltiplos instantes são coletadas. Outra opção seria descartar completamente a leitura inteira, o que só seria plausível caso não haja a expectativa de resposta do modelo para cada leitura da entrada (o que foi perguntado durante a identificação do problema). Por fim, há também as opções de completar os valores faltantes do treino e da inferência usando o próprio Chronos 2, o que diminuiria ainda mais a influência dos dados no modelo final (apontando para um desempenho ruim) e a opção de interpolar os valores faltantes (contando ou não com a esparsidade natural dos dados) o que precisaria ser definido como possível com a ajuda de profissionais especialistas nos sensores.

Com relação à robustez, métricas como desvio padrão e box-plots ajudam a verificar quão padronizados os dados são e quantos outliers há em cada variável. Também é importante verificar com especialistas em MP sobre métricas de ruído específicas para os sensores analisados, além de métricas menos sensíveis a ruído e outliers. Em caso de dados muito ruidosos, o uso de janelas deslizantes, extraíndo métricas robustas de cada feature, também é uma opção.

Sobre a tipagem, a única variável com tipo não numérico (excluindo o tempo, que é descartado em favor da modelagem sequencial) é COM, que pode ser facilmente mapeada para uma variável numérica binária compatível com todos os modelos utilizados. Também é importante verificar com especialistas o escopo das variáveis, se há variáveis cíclicas (como por exemplo ângulo, onde $0^\circ = 360^\circ$), que requeiram remodelagem ou variáveis de progressão não-linear que usualmente são tratadas

segundo algum critério (tirando log, elevando ao quadrado, etc). Por fim, a quantidade final de dados vai definir quais próximos modelos serão viáveis, ou se é necessário a obtenção de mais dados de outras fontes.

Outros insights necessários de se obter sobre os dados são se as variáveis de cada máquina (compressor e turbina) são independentes, tanto contextualmente (perguntando a especialistas) quanto estatisticamente (usando métricas como correlação linear, Coeficiente de Informação Mútua, etc), ou se podem ser usadas como features para as duas previsões.

Por fim, as decisões de modelagem devem ser documentadas (para posterior implementação dos pipelines de coleta e inferência pela equipe de engenharia de dados) e datasets devem ser criados de acordo com o tratamento dos dados. Três modelagens possíveis para os dados foram sugeridas na revisão e ao longo desta sessão: Modelagem direta da sequência, concatenando todos os sensores; modelagem por blocos contendo uma leitura de sensor cada, gerando features multi-sensor quantizadas por intervalos de 4, 8 ou 12 segundos, ou janelas deslizantes de intervalos maiores, aplicando métodos de feature extraction para extração de informação mais importante (nova revisão da literatura em técnicas de feature extraction seria necessária).

4.4. Organização das hipóteses

Conforme mencionado, as hipóteses a serem testadas inicialmente serão todas utilizando o Chronos, que tem o melhor custo-eficiência-benefício para o problema, e é compatível com todas as modelagens dos dados propostas. Daremos prioridade para vertentes zero-shot, já que não requerem treinamento, em comparação a vertentes que fazem fine-tuning. Além disso, como o modelo é especializado para forecasting, podemos modelar a regressão do RUL adicionando ele como feature à sequência e realizando o forecasting da sequência inteira, o que exclui a necessidade de extração dos embeddings da sequência. Por fim, podemos testar posteriormente variantes que exijam mais técnicas de engenharia de features, como as baseadas em janela deslizante, em favor das variantes com modelagem mais simples. Fechamos a lista de hipótese em ordem de simplicidade com:

- Dados sequenciais + zero-shot forecasting da sequência e do RUL com Chronos 2;
- Dados em blocos + zero-shot forecasting da sequência e do RUL com Chronos 2;
- Dados sequenciais + fine-tuning para forecasting da sequência e do RUL com Chronos 2;
- Dados em blocos + fine-tuning para forecasting da sequência e do RUL com Chronos 2;
- Dados sequenciais + fine-tuning para regressão/classificação a partir da sequência com Chronos 2;

- Dados em blocos + fine-tuning para regressão/classificação a partir da sequência com Chronos 2;
- Dados em janelas deslizantes + engenharia de features + zero-shot forecasting da sequência e do RUL com Chronos 2;
- Dados em janelas deslizantes + engenharia de features + fine-tuning para forecasting da sequência e do RUL com Chronos 2;
- Dados em janelas deslizantes + engenharia de features + fine-tuning para regressão/classificação a partir da sequência com Chronos 2;
- Outros modelos.

5. Conclusão

Com este trabalho, fui capaz de propor não só um plano de ação para a solução do problema proposto voltado ao contexto corporativo de produção em equipe de soluções em software e ciência de dados, como também apresentei múltiplas propostas de modelagem e solução do problema usando técnicas estado-da-arte na atualidade, tanto em desempenho quanto em eficiência e plausibilidade de implementação. Trabalhos futuros podem explorar o detalhamento de outras partes do plano de ação, bem como a execução real do plano em um contexto de produção real na Dynamox.

6. Bibliografia

1. Amazon-Science (2025) *Amazon-Science/Chronos-forecasting: Chronos: Pretrained models for Probabilistic Time Series forecasting*, GitHub. Available at: <https://github.com/amazon-science/chronos-forecasting> (Accessed: 21 October 2025).
2. Data Science Project Management (2025) *Domino Data Lab*. Available at: <https://domino.ai/resources/field-guide/managing-data-science-projects> (Accessed: 21 October 2025).
3. *Guia do IBM SPSS modeler crisp-DM IBM* (2023) *IBM Documentation*. Available at: https://www.ibm.com/docs/pt-br/SS3RA7_18.5.0/nl/pt/BR/pdf/ModelerCRISP_DM.pdf (Accessed: 21 October 2025).
4. Namuduri, S. *et al.* (2020) 'Review—deep learning methods for sensor based predictive maintenance and future perspectives for electrochemical sensors', *Journal of The Electrochemical Society*, 167(3), p. 037552. doi:10.1149/1945-7111/ab67a8.
5. Sumner, S. (2025) *Plan for AI Adoption - Cloud Adoption Framework*, *Cloud Adoption Framework | Microsoft Learn*. Available at:

<https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/scenarios/ai/plan> (Accessed: 21 October 2025).

6. Valente, M.T. (2020) *Engenharia de Software Moderna*. Belo Horizonte, Minas Gerais: Independente.
7. Vishwas, B.V. and Macharla, S.R. (2025) 'Chronos: Pre-trained Probabilistic Time Series model', *Time Series Forecasting Using Generative AI*, pp. 155–167. doi:10.1007/979-8-8688-1276-7_5.
8. Xu, G. et al. (2019) 'Data-driven fault diagnostics and prognostics for Predictive Maintenance: A brief overview', *2019 IEEE 15th International Conference on Automation Science and Engineering (CASE)* [Preprint]. doi:10.1109/coase.2019.8843068.
9. Khan, N.A. and Ali, S. (2022) 'Robust sparse reconstruction of signals with gapped missing samples from multi-sensor recordings', *Digital Signal Processing*, 123, p. 103392. doi:10.1016/j.dsp.2022.103392.
10. Khan, U. et al. (2025) 'A comprehensive survey on Deep Learning-based predictive maintenance', *ACM Transactions on Embedded Computing Systems* [Preprint]. doi:10.1145/3732287.